Reporte de Despliegue

Ruta de Inducción (Ruta de Inducción / universidad-g-max) — Stack nuevo Django + React

Fecha del despliegue	06 de octubre de 2026
Documento	ReporteDespliegueRytaInduccion06-oct
Propósito	Manual de documentación del despliegue, para un programador ajeno al proyecto
Rama / commit	`segundaSimple` — HEAD `7da92ae`
Servidor (VPS)	`root@177.7.49.5` — AlmaLinux 9.7 — Hostinger `srv1630508.hstgr.cloud`
Dominio	`https://www.megas.com.co` (también responde el apex `megas.com.co`)
Alcance	Despliegue del stack nuevo (backend Django + frontend React + auth Firebase) en paralelo al stack antiguo
Estado final	**TODAS las rutas verificadas responden OK** — despliegue operativo
Secretos	Este documento NO contiene credenciales sensibles; cita solo nombres de variables

Índice

1. Resumen ejecutivo
2. Contexto del proyecto
3. Inventario de infraestructura resultante
4. Modelo de tráfico (cómo llega una petición)
5. Trazabilidad: cambios de código (commits)
6. Bitácora del despliegue fase por fase
7. Verificación end-to-end (resultados medidos)
8. Runbook de operación (para el próximo programador)
9. Mapa de URLs y endpoints
10. Configuración requerida (.env.produccion)
11. Hallazgos del entorno y trampas conocidas
12. Estado final de los datos (DB)
13. Pendientes para el equipo / usuario
14. Apéndices

> Convención de rutas: `www.megas.com.co` es el dominio canónico (redirige desde `megas.com.co` y desde http). En todo el documento, los comandos sin usuario explícito se ejecutan como `root` en el VPS `177.7.49.5`.

1. Resumen ejecutivo

Durante el 06-oct-2026 se desplegó en el VPS el **stack nuevo** de la Ruta de Inducción, correspondiente a la rama `segundaSimple` del repositorio `github.com/marketMegas/dockerRutaInduccion`. El stack nuevo reemplaza a nivel de dominio la funcionalidad del sitio `/rutainduccion/` (SPA React + API Django con administración) sin tocar el stack antiguo (`/rutadeexpertos/` , que sigue activo).

Resultado (verificado el mismo día)

Ruta pública	Resultado	Responsable
`/rutainduccion/` (SPA React)	200	Frontend nuevo (contenedor `induccion-megas-frontend` , 5138)
`/api/*` (API Django)	200	Backend nuevo (`ruta-induccion-backend` , 8001)
`/admin/` (panel Django)	200	Backend nuevo (8001)
`/static/*` (estáticos Django/WhiteNoise)	200	Backend nuevo (8001)
`/rutadeexpertos/`	200	Stack antiguo (5137 / 8000) — no modificado
`http://.../*`	301 → https	nginx del host (puerto 80)

Decisión de arquitectura tomada en el día

- Servir mobiliario nuevo y viejo en paralelo desde el mismo VPS y el mismo dominio, conmutado por prefijo de URL.

- Los puertos de los contenedores nuevos se publicaron **solo en loopback** (`127.0.0.1`); el TLS lo termina OpenLiteSpeed (443) y el redirect http→https lo hace nginx (80). Nunca exponer 5137/5138/8000/8001 al exterior.
- Base de datos restaurada desde el backup `db.sqlite3.bak-antes-borrar-autoglp` (la más completa).
- El mecanismo efectivo de ruteo/proxy en este servidor resultó ser el bloque `rewrite` con `[P]` del `.htaccess` (no los `context proxy` de LiteSpeed; ver sección 6.F y 11).

2. Contexto del proyecto

2.1 Repositorio

- Ruta local (Windows): `C:\Users\DESARROLLO MEGAS\Documents\dockerRutaInduccionMEJORADO\universidad-g-max - copiaENSAYANDO`
- Rama productiva: `segundaSimple` — remote: `https://github.com/marketMegas/dockerRutaInduccion.git`
- Commit desplegado: `7da92ae` (último de la serie del día; ver sección 5).
- El mismo repositorio está clonado en el VPS en `/root/induccion-megas` (misma rama).

2.2 Stack del proyecto

Capa	Tecnología	Detalle
Backend	Django (Python 3.12-slim)	Apps: `usuarios` , `courses` , `evaluaciones` , `foro` . WSGI con gunicorn en `:8000` .
Frontend	React + Vite (Node 20) sobre nginx:alpine	SPA compilado con `base '/rutainduccion/'` , bundle `assets/index-YccBd-tK.js` .
Auth	Firebase Authentication (Email/Password)	Config embebida en `frontend/src/config/firebase.js` ; backend audita los ID tokens con el SDK de Firebase Admin.
Autorización	Whitelist en la DB de Django	`usuarios/permisos.py::usuario_autorizado(email)` contra `auth_user` .
DB	SQLite	Archivo `/root/induccion-megas/data/db.sqlite3` (montado por volumen en el backend).
Proxy / TLS	OpenLiteSpeed 1.8.5 (443) + nginx del host (80)	Ver modelo de tráfico (sección 4).

2.3 Árbol de URLs (rutas de la SPA)

- **Públicas**: `/rutainduccion/login` , `/rutainduccion/register` , `/rutainduccion/foro` , recursos.
- **Sesión de Firebase** (sin alta en Django): foro, recursos — `AuthenticatedRoute` .
- **Sesión de Firebase + alta en Django** (whitelist): cursos, lecciones, evaluaciones, certificado, calificaciones — `ProtectedRoute` .
- El panel de administración `/admin/` es el admin estándar de Django (independiente de Firebase).

3. Inventario de infraestructura resultante

3.1 Sistema operativo y servicios base (VPS)

Componente	Versión / detalle
SO	AlmaLinux 9.7
Docker	Client Docker 29.4.1; binario de compose reportado como "Docker Compose version v5.1.3" (no estándar; funciona con `--env-file`)
OpenLiteSpeed	LiteSpeed/1.8.5 Open — escucha en `:443`
nginx (sistema)	nginx/1.20.1 — escucha en `:80` (único dueño del 80)
Firewall	firewalld — solo 80/443 abiertos al exterior (los puertos Docker quedan en loopback)

3.2 Contenedores (estado final)

Contenedor	Imagen	Publicación	Restart	Stack
`ruta-induccion-backend`	`induccion-megas-backend:8de9757`	`127.0.0.1:8001→8000`	unless-stopped	Nuevo
`induccion-megas-frontend`	`induccion-megas-frontend:7da92ae`	`127.0.0.1:5138→80`	unless-stopped	Nuevo
`universidadg-max-frontend-1`	`universidadg-max-frontend`	`0.0.0.0:5137→80`	no	Antiguo (rutadeexpertos)
`universidadg-max-backend-1`	`universidadg-max-backend`	`0.0.0.0:8000→8000`	no	Antiguo (rutadeexpertos)

3.3 Imágenes Docker disponibles (importante para rollback)

Imagen	Antigüedad	Rollback
`induccion-megas-frontend:7da92ae`	6-oct	nueva (ok)
`induccion-megas-backend:8de9757`	6-oct	nueva (ok)
`induccion-megas-backend:b3d27e5`	6-oct	build intermedio
`induccion-megas-backend:latest`	agosto	**imagen vieja del stack nuevo** (no usar como referente de "último")
`induccion-megas-frontend:latest`	agosto	imagen vieja del stack nuevo
`universidadg-max-backend` / `frontend`	agosto	stack antiguo (rutadeexpertos)

> En este proyecto el tag `:latest` NO es la última versión: las imágenes se etiquetan por commit (`APP_VERSION`). Un `docker compose up` sin `APP_VERSION` exportada re-cae a `:latest` (vieja).

3.4 Red Docker y datos persistentes

- Red: `induccion-megas_default` (creada por el proyecto compose fijo `name: induccion-megas`). El backend responde en la red interna como `backend:8000` (usado por el nginx del contenedor frontend).

- Datos del backend (volúmenes bind, viven FUERA del contenedor y del repo):

```
/root/induccion-megas/data/db.sqlite3      # base de datos SQLite productiva
/root/induccion-megas/media/               # certificados PDF emitidos
            └── Certificados_Emitidos/     # 4 PDFs (06-oct)
```

3.5 Backups generados durante el despliegue

```
/root/backups/pre-deploy-20261006-134248/
   ├── db.sqlite3-live-autoglp              # backup de la DB VIVA antes del cambio
   ├── .env.produccion                      # config productiva previa
   └── media/Certificados_Emitidos/
        ├── Certificado_Ga_BemCjWm.pdf
        └── Certificado_juan.pdf

Otros backups puntuales creados en el día:
   /root/induccion-megas/db.sqlite3.bak-antes-borrar-autoglp     # base restaurada (fuente)
   /root/induccion-megas/db.sqlite3.bak-antes-evaluaciones       # alternativa descartada
   /usr/local/lsws/conf/vhosts/www.megas.com.co/vhost.conf.pre-deploy.20261006-140332
   /home/www.megas.com.co/public_html/.htaccess.pre-deploy.*
   /etc/nginx/nginx.conf.pre-deploy.*
```

4. Modelo de tráfico (cómo llega una petición)



4.1 Puntos clave del modelo

- **Puerto 80**: solo nginx del sistema; redirige todo a https (301 al dominio canónico). El listener :80 de LiteSpeed quedó inactivo (nginx se quedó con el bind).

- **Puerto 443**: OpenLiteSpeed con dos vhosts (`www.megas.com.co` y `megas.com.co`) que comparten el mismo `docRoot` (`/home/www.megas.com.co/public_html`).
- **El ruteo efectivo son los rewrites `[P]` del `.htaccess`** (auto-cargado, sin reinicio de LiteSpeed). Los `context proxy` declarados en los vhost.conf (se añadieron en el día, ver 6.F) **no dispararon** en esta instalación; por eso la solución real quedó en el `.htaccess` .
- El `.htaccess` es compartido por ambos vhost (mismo public_html); al ser `autoLoadHtaccess 1` y leerse en vivo, los cambios se aplican sin reiniciar LiteSpeed.
- Los contenedores nuevos NO se exponen: LiteSpeed/nginx del host los alcanzan por `127.0.0.1` . Los contenedores viejos (rutadeexpertos) sí publican en `0.0.0.0` , pero el firewall solo deja pasar 80/443.
- **X-Forwarded-Proto**: el proxy de LiteSpeed envía `X-Forwarded-Proto: https` y `X-Forwarded-Host` al backend (verificado con sonda). Django lo honra con `SECURE_PROXY_SSL_HEADER = ('HTTP_X_FORWARDED_PROTO','https')` , así los correos y certificados generan URLs https.

> El contenedor frontend tiene su PROPIO `nginx.conf` (archivo `nginx.conf` en la raíz del repo) que, además de servir la SPA, vuelve a proxear `/api/` , `/admin/` y `/static/` hacia `backend:8000` por la red interna. Es el "plan B": si algún día LiteSpeed apunta el frontend delante de todo, el proxy interno sigue funcionando. Actualmente la entrada es LiteSpeed → `.htaccess [P]` → cada contenedor.

5. Trazabilidad: cambios de código (commits)

Hoy se crearon y pushearon a `segundaSimple` los siguientes commits (en orden cronológico; HEAD final `7da92ae`):

Commit	Archivos clave	Qué cambió y por qué
`ee5d6cf`	`filesParaDeploy/docker-compose.prod.yml`	Backend: `container_name: ruta-induccion-backend` y puerto `127.0.0.1:8001:8000` (en vez de `expose`). Frontend: puerto `127.0.0.1:5138:80`. Se retiró del frontend el bind de `media` (los certificados deben servir desde el backend).
`b3d27e5`	`filesParaDeploy/docker-compose.prod.yml`	`build: context: ..` para ambos servicios: el Dockerfile necesita la raíz del repo (backend copia `.`, frontend copia `frontend/`).
`8de9757`	`filesParaDeploy/docker-compose.prod.yml`	`name: induccion-megas` a nivel de proyecto compose: fuerza la red `induccion-megas_default`, que ya existía en el VPS (huérfana del intento anterior). Reutilizarla evita que compose cree una red nueva y deje frontend y backend aislados.
`7da92ae`	`frontend/src/pages/{LoginPage,RegisterPage,DashboardPage}.jsx`	Fix del logo: import `../../../nuevoLOGOMegas.png` → `../../nuevoLOGOMegas.png`. El PNG vive en `frontend/`, y en el build del contenedor el `frontend/` queda como raíz de módulos; la ruta con tres `../` rompía el build del Dockerfile (no así en local).

5.1 Archivos modificados locales SIN commitear (importante)

```
$ git status -s
 M .env.produccion                          # env local de desarrollo (no debe comitearse)
 M frontend/src/pages/CalificacionesPage.jsx  # versión local sin Firestore; NO se desplegó
```

El bundle desplegado (hash `index-YccBd-tK.js`) corresponde exactamente a `7da92ae` (comiteado), y NO incluye la versión local de `CalificacionesPage.jsx`. Por eso el hash no coincide con un build local si el equipo tiene la versión modificada en disco (diferencias de Firestore).

6. Bitácora del despliegue fase por fase

Fase A — Ajustes del compose y push (local)

- Se editó `filesParaDeploy/docker-compose.prod.yml` con los cambios de la sección 5 (commits `ee5d6cf`, `b3d27e5`, `8de9757`, `7da92ae`).
- Se pusheó la rama `segundaSimple`.

Fase B — Backup preventivo en el VPS

- Antes de tocar nada se copió: la DB viva (`db.sqlite3-live-autoglp`), el `.env.produccion` y `media/` (2 certificados) a `/root/backups/pre-deploy-20261006-134248/` .
- Es el punto de restauración del estado previo (sección 3.5).

Fase C — Selección y restauración de la base de datos

- Había dos candidatas en el VPS: `db.sqlite3.bak-antes-borrar-autoglp` y `db.sqlite3.bak-antes-evaluaciones` .
- Se eligió la primera por ser la más completa (incluía datos de evaluaciones y foro). Se restauró como `/root/induccion-megas/data/db.sqlite3` .
- Se copió `media/` (certificados) a `/root/induccion-megas/media/` .
- Conteos de la base restaurada: 22 tablas, 27 migraciones; ver conteos finales tras migrar en la sección 12.
- Nota: existen 3 filas `auth_user` duplicadas para `marketingmagas2026@gmail.com` ; inofensivas porque `usuario_autorizado()` usa `.filter(email__iexact, is_active=True).order_by('id').first()` .

Fase D — Deploy del backend

1. `git pull` en `/root/induccion-megas` hasta `7da92ae` .
2. **Incidente DNS en build**: `docker compose build` falló por DNS (pip) → se construyó la imagen con la red del host:

```
docker build --network host -t induccion-megas-backend:8de9757 -f Dockerfile.backend .
```

3. **Incidente restart con resta de iptables**: tras un `systemctl restart docker` el `ps` dejó de publicar puertos (cadenas DOCKER-FORWARD purgadas por firewalld). Se resolvió con otro `systemctl restart docker` y arranque manual de los contenedores con `restart=no` (los dos nuevos son `unless-stopped` y arrancan solos).
4. **Incidente de red compose**: como el proyecto compose había cambiado de nombre vs. el intento anterior, no encontraba la red; la solución fue el `name: induccion-megas` (commit `8de9757`) para reutilizar `induccion-megas_default` .
5. **Incidente de tag**: un `up` sin `APP_VERSION` exportado montó la imagen vieja `:latest` (que además no levantaba). Se exporta siempre:

```
export APP_VERSION=$(git rev-parse --short HEAD)
```

6. Arranque real:

```
docker compose --env-file .env.produccion -f filesParaDeploy/docker-compose.prod.yml up -d --no-deps bac
```

7. Validador tras levantar: `docker exec ruta-induccion-backend python manage.py migrate --noinput` completó TODAS las migraciones, incluidas las nuevas `courses.0008_certificado_token` y `courses.0009_backfill` .
8. Pruebas directas en `127.0.0.1:8001` : `/admin/` 302, `/static/admin/css/base.css` 200, `/api/foro/temas/` 200 (JSON).

Fase E — Deploy del frontend

1. El build local del contenedor fallaba por el import del logo → fix en 3 páginas (commit `7da92ae`).

2. Build con red del host por el mismo problema de DNS:

```
docker build --network host -t induccion-megas-frontend:7da92ae -f Dockerfile.frontend .
export APP_VERSION=$(git rev-parse --short HEAD)
docker compose --env-file .env.produccion -f filesParaDeploy/docker-compose.prod.yml up -d --no-deps fr
```

3. Se eliminaron el contenedor y la imagen viejos del frontend anterior (`ruta-induccion-frontend` , `induccion-megas-frontend:latest` quedó como imagen para rollback).
4. Validación: `http://127.0.0.1:5138/rutainduccion/` sirve la SPA (200) y, gracias al `nginx.conf` interno, proxea correcto `/api/` , `/admin/` y `/static/` → `backend:8000` (probado con `Host: www.megas.com.co`).

Fase F — Routing externo (la parte más delicada)

1. Estado previo: `/api/` se resolvía por una regla rota (`backend_api` apuntando a un extprocessor inexistente, 500), y el frontend viejo tomaba `/admin/` y `/static/` por un contexto anidado defectuoso (404).
2. Primer intento: editar `/usr/local/lsws/conf/vhosts/www.megas.com.co/vhost.conf` agregando `context /api/` , `context /admin/` y `context /static/` de tipo `proxy` con handler `dockerBackend_Induccion` + `lswsctrl restart` . Resultado: **404** (LiteSpeed no honró los contexts).
3. Segundo intento: replicar los mismos contexts en el segundo vhost (`megas.com.co/vhost.conf`). Resultado: **404** igual.
4. Diagnóstico con el cuerpo de las respuestas: `/admin/login/` devolvía la página de login de Django y `/api/foro/temas/` el JSON real del backend NUEVO en cuanto se toparon. El 404 era cuestión de *cuándo* se aplicaba la config: los cambios en `.htaccess` se leen en vivo.
5. **Solución efectiva**: rewrites proxy `[P]` en `/home/www.megas.com.co/public_html/.htaccess` apuntando a los extprocessors por nombre (mismo mecanismo que ya usaba `/rutainduccion` y `/rutadeexpertos`):

```
# 4. Backend nuevo de la Ruta de Induccion: API, admin y estaticos
RewriteRule ^/?api/(.*)$    http://dockerBackend_Induccion/api/$1   [P,L]
RewriteRule ^/?admin/(.*)$  http://dockerBackend_Induccion/admin/$1 [P,L]
RewriteRule ^/?static/(.*)$ http://dockerBackend_Induccion/static/$1 [P,L]
```

   Al ser `autoLoadHtaccess 1` , se aplica en vivo sin reiniciar. Resultado: **todas 200**.
6. Aclaración arquitectónica: en esta instalación los contexts proxy de LiteSpeed no maduran; la vía fiable (y ya usada previamente) son los rewrites `[P]` con la dirección `http://<nombre_extprocessor>/...` , donde el host del rewrite coincide con el nombre de un extprocessor declarado a nivel de servidor en `httpd_config.conf` .
7. Se limpió el `.htaccess` de reglas muertas: `backend_api` , regla antigua `/api/` y `/api-induccion` . Backup `.htaccess.pre-deploy.*` .
8. nginx del host (puerto 80): se reemplazó el server block por defecto por un redirect total a https (`return 301 https://www.megas.com.co$request_uri`), preservando `/.well-known/acme-challenge/` hacia `/usr/local/lsws/Example/html` (webroot compartido con litespeed para renovación de certificados). `nginx -t` OK + `systemctl reload nginx` .
9. **Verificación "sonda"**: se escribió un mini servidor HTTP en `127.0.0.1:8100` , un extprocessor `probe` temporal y una regla rewrite temporal para inspeccionar las cabeceras que LiteSpeed envía al backend a través del rewrite `[P]` . El log mostró:

```
PATH=/probex/zz/aa
host: www.megas.com.co
X-Forwarded-Host: www.megas.com.co
X-Forwarded-Proto: https
X-Forwarded-For: 177.7.49.5
```

→ Confirma que el backend recibe **https** en `X-Forwarded-Proto`; con el `SECURE_PROXY_SSL_HEADER` de Django las URLs absolutas (correos, certificados, CSRF) se generan como https. La sonda se retiró al terminar (extprocessor y regla eliminados, proceso cerrado, archivos temporales borrados).

Fase G — Verificación end-to-end

Ver tabla completa en la sección 7.

Fase H — Validación de Firebase sin consola

- El equipo no dispone de acceso a Firebase Console. Se validó por la API pública de Identity Toolkit (solo con la API key, que es pública y va embebida en el bundle del frontend):
  - `accounts:signInWithPassword` con un password incorrecto devolvió `INVALID_LOGIN_CREDENTIALS` (y NO un error de proveedor deshabilitado) → el proveedor Email/Password está **activo**.
  - `accounts:createAuthUri` respondió `sessionId` (flujo válido).
- Conclusión: **no se requiere añadir dominios a Authorized domains** para este flujo. Esa restricción aplica a OAuth popup/redirect y a email-action-links; `signInWithEmailAndPassword` (lo que usa la app) no queda bloqueado por el dominio. El dominio autorizado `plataforma-gmax.firebaseapp.com` y la API key de `frontend/src/config/firebase.js` son reales y quedaron embebidas en el bundle `index-YccBd-tK.js`.

7. Verificación end-to-end (resultados medidos)

7.1 Rutas HTTPS — dominio `www.megas.com.co`

URL	Código	Comentario
`/rutainduccion/`	200	SPA; HTML referencia `assets/index-YccBd-tK.js` (coincide con el contenedor)
`/api/foro/temas/`	200	JSON con 3 temas (ids 1..3)
`/api/auth/correo-autorizado/` GET	405	ruta correcta (solo POST)
`/api/auth/correo-autorizado/` POST `{email}`	200	`{"autorizado": bool}`
`/api/auth/correo-autorizado/` POST sin email	400	`{"error":"Falta el correo."}`
`/admin/login/`	200	login de Django
`/static/admin/css/base.css`	200	WhiteNoise vía backend
`/api/cursos/certificado/aaa/bbb/ccc/`	404	ruta existe; token inexistente → 404 (comportamiento esperado)
`/rutadeexpertos/`	200	stack antiguo intacto
`https://megas.com.co/rutainduccion/` (apex)	200	el cert cubre ambos dominios

7.2 Redirects HTTP → HTTPS (nginx del host)

Request	Código	Location
`http://megas.com.co/`	301	`https://www.megas.com.co/`
`http://www.megas.com.co/`	301	`https://www.megas.com.co/`
`http://www.megas.com.co/rutainduccion/`	301	`https://www.megas.com.co/rutainduccion/`

7.3 Verificaciones internas (dentro del VPS)

- `127.0.0.1:8001/admin/` → 302 (login). Directo al backend.
- `127.0.0.1:8001/api/foro/temas/` → 200 directo.
- `127.0.0.1:5138` con `Host: www.megas.com.co` → SPA 200 y proxy interno `/api/`, `/admin/`, `/static/` → 200.
- Sonda de cabeceras (ver 6.F): LiteSpeed envía `X-Forwarded-Proto: https`.

8. Runbook de operación (para el próximo programador)

Todos los comandos siguientes se ejecutan como `root` en el VPS y el repositorio está en `/root/induccion-megas`.

8.1 Rebuild y despliegue de una versión nueva

```
cd /root/induccion-megas
git fetch origin && git checkout segundaSimple && git pull
git checkout <commit-or-rama>        # opcional: fijar una versión
export APP_VERSION=$(git rev-parse --short HEAD)
# El DNS del daemon a veces falla para pip/npm; si falla el build, usar --network host:
docker compose --env-file .env.produccion -f filesParaDeploy/docker-compose.prod.yml build
#  o, alternativamente:
# docker build --network host -t induccion-megas-backend:$APP_VERSION   -f Dockerfile.backend  .
# docker build --network host -t induccion-megas-frontend:$APP_VERSION -f Dockerfile.frontend .

docker compose --env-file .env.produccion -f filesParaDeploy/docker-compose.prod.yml up -d --no-deps backen
docker compose --env-file .env.produccion -f filesParaDeploy/docker-compose.prod.yml up -d --no-deps fronte
```

> SIEMPRE exportar `APP_VERSION`. Sin ella, compose usa `:latest` y levanta imágenes viejas (re-gresivo).

8.2 Rollback a una versión anterior

Las imágenes se etiquetan por commit; reconstruir/montar un tag anterior compuesto (commits de build: backend usa la imagen del código del backend del commit `<B>`, frontend de `<F>`):

```
docker build --network host -t induccion-megas-backend:<commitB>  -f Dockerfile.backend  .   # checkout de
docker build --network host -t induccion-megas-frontend:<commitF> -f Dockerfile.frontend .
# y volcar la imagen vieja con el mismo procedimiento del 8.1.
```

8.3 Logs

```
docker logs --tail 200 -f ruta-induccion-backend
docker logs --tail 200 -f induccion-megas-frontend
tail -f /usr/local/lsws/logs/error.log          # errores de LiteSpeed
```

8.4 Backup / restauración de datos (lo único que se backupea)

```
mkdir -p /root/backups/`date +%Y%m%d-%H%M%S`
cp -a /root/induccion-megas/data/db.sqlite3   /root/backups/<ts>/
cp -a /root/induccion-megas/media             /root/backups/<ts>/
# restaurar: parar el backend, copiar, subir de nuevo
docker restart ruta-induccion-backend
```

> La DB es SQLite: al respaldar en caliente conviene un `cp` atómico (o `sqlite3 db .backup`).

8.5 Reinicio de la máquina (containers que no auto-arrancan)

- Los dos contenedores nuevos son `restart: unless-stopped` → arrancan solos con el daemon.
- Los dos contenedores viejos (`universidadg-max-*-1`) son `restart=no` → levantarlos a mano tras un reboot:

```
systemctl restart docker            # a veces necesario tras reinicios: restaura iptables de docker (firewa
docker start universidadg-max-frontend-1 universidadg-max-backend-1
```

8.6 Cambios de ruteo (el mecanismo que SÍ funciona)

- Editar `/home/www.megas.com.co/public_html/.htaccess` (rewrites `[P]`). Se aplica EN VIVO; no hace falta reiniciar.
- Si se añade un destino nuevo, declarar antes su extprocessor en `/usr/local/lsws/conf/httpd_config.conf` y reiniciar LiteSpeed (`/usr/local/lsws/bin/lswsctrl restart`).

8.7 Verificación rápida post-cambio

```
curl -s -o /dev/null -w '%{http_code}\n' https://www.megas.com.co/rutainduccion/
curl -s -o /dev/null -w '%{http_code}\n' https://www.megas.com.co/api/foro/temas/
curl -s -o /dev/null -w '%{http_code}\n' https://www.megas.com.co/admin/login/
curl -s -o /dev/null -w '%{http_code}\n' https://www.megas.com.co/static/admin/css/base.css
curl -s -o /dev/null -w '%{http_code}\n' http://www.megas.com.co/            # 301 esperado
```

9. Mapa de URLs y endpoints

9.1 Públicas (externas)

URL	Qué es
`/rutainduccion/` y `/rutainduccion/*`	SPA React (login, register, cursos…)
`/rutadeexpertos/` y `/rutadeexpertos/*`	Stack antiguo (NO migrado todavía)
`/admin/`	Admin Django (superuser de Django)
`/static/*`	Estáticos de Django (WhiteNoise)
`/api/*`	API JSON (foro, cursos, evaluaciones, auth)

9.2 Endpoints internos del backend

```
# core_project/urls.py
/admin/                                                    → admin.site.urls
/ (incluye usuarios.urls)
/api/foro/          → foro.urls
/api/cursos/        → courses.urls
/api/cursos/evaluaciones/ → evaluaciones.urls

# usuarios/urls.py (destacados)
/api/auth/correo-autorizado/ (POST {"email": "…"})  → {"autorizado": bool}   [sin sesión requerida]
/api/auth/verificar_acceso   (POST Bearer ID token) → valida sesión Firebase contra whitelist
# courses/urls.py (destacados)
/api/cursos/enviar-certificado/                    (POST multipart, con auth) → registra el PDF y envía el
/api/cursos/certificado/<user_id>/<course_id>/<token>/  → descarga del certificado PDF (única con token, si
```

> `/media/` NO se expone públicamente a propósito: los certificados se descargan por el endpoint tokenizado arriba. La regla `location /media/` del nginx del contenedor frontend es un plan B interno, no una puerta pública.

9.3 Config de los servicios en el compose

- Backend: comando de arranque `migrate --noinput` → `collectstatic --noinput` → `gunicorn :8000`.
- Volúmenes: `data/db.sqlite3 → /app/db.sqlite3` y `media → /app/media`.
- `logging: json-file` con `max-size 10m` / 3 archivos rotados (evita llenar el disco).

10. Configuración requerida (.env.produccion)

Solo se listan los NOMBRES y su propósito (los valores con secretos NO se publican; viven en `filesParaDeploy/.env.produccion` del repo clonado y también en `/root/induccion-megas/.env.produccion`).

Variable	Obligatoria	Propósito
`SECRET_KEY`	Sí (`:?`)	Secret de Django. Sin ella el contenedor no arranca.
`DEBUG`	No (default False)	Nunca True en prod.
`ALLOWED_HOSTS`	No	`megas.com.co,www.megas.com.co,localhost,127.0.0.1,177.7.49.5`
`CSRF_TRUSTED_ORIGINS`	No	`https://megas.com.co,https://www.megas.com.co`
`CORS_ALLOWED_ORIGINS`	No	Vacío por defecto: el frontend usa rutas relativas, sin CORS.
`EMAIL_HOST_USER`	Sí (`:?`)	Cuenta SMTP que manda los correos (certificados, recuperación de password).
`EMAIL_HOST_PASSWORD`	Sí (`:?`)	Password SMTP.
`NOTIF_DESTINATARIOS`	Sí (`:?`)	Correos internos de aviso (equipo).
`FIREBASE_PROJECT_ID`	Sí (`:?`)	Id del proyecto Firebase (`plataforma-gmax`). Sin él, el backend no audita tokens y toda la API da 503.

En `core_project/settings.py` : `SECURE_PROXY_SSL_HEADER = ('HTTP_X_FORWARDED_PROTO', 'https')` , `MEDIA_URL = '/media/'` , `MEDIA_ROOT = BASE_DIR / 'media'` .

11. Hallazgos del entorno y trampas conocidas

- **LiteSpeed no aplica** `context proxy` en esta instalación (probado en ambos vhosts). El ruteo por proxy efectivo son los rewrites `[P]` con host = nombre de extprocessor. Los contexts añadidos hoy quedaron colocados en los vhost.conf (inactivos) — limpieza estética opcional.
- **vhost doble**: `www.megas.com.co` y `megas.com.co` comparten docRoot; por eso el `.htaccess` es válido para ambos. En el vhost `megas.com.co` , los contexts /api-//admin//static quedaron anidados dentro del bloque del context acme-challenge (defecto estético; sin impacto porque el ruteo real está en el .htaccess). No intentar "arreglarlo" sin saber que los contexts no son el mecanismo vivo.
- **IPtables / firewalld**: tras `systemctl restart docker` (o un reinicio) a veces faltan las cadenas DOCKER-FORWARD y los puertos dejan de publicarse. Remedio: reiniciar docker otra vez y `docker start` los contenedores `restart=no` .
- **DNS en builds**: pip/npm dentro de `docker build` falló; usar `--network host` .
- **Compose "v5.1.3"**: binario no estándar pero funcional; acepta `--env-file` . No confiar en etiquetas por defecto.
- `:latest` **es engañoso**: es la imagen de otro intento. Etiquetar SIEMPRE por `APP_VERSION` .
- **Emails duplicados** en `auth_user` (3× marketingmagas2026@gmail.com): inofensivos por el `filter().order_by('id').first()` .
- **El** `nginx.conf` **de la raíz del repo** es el del CONTENEDOR frontend (`COPY nginx.conf /etc/nginx/conf.d/default.conf`). Ojo: existe también `frontend/nginx.conf` (legacy) y `filesParaDeploy/nginx-host.conf` (borrador de host).
- **El tag** `8de9757` **del backend** es el comit del build correcto del backend (el código del backend no cambió en `7da92ae` , ese commit solo tocó el frontend).
- **Env del frontend corre a través de litespeed**: el proxy de litespeed manda `X-Forwarded-Proto/Host/For` ; el nginx interno del contenedor los reenvía sin pisarlos (usa `$http_x_forwarded_proto` con fallback `$scheme`).
- **Puerto 80 lo tiene nginx** (no liteSpeed); `ss -lntp` lo confirma: `nginx 0.0.0.0:80` , `litespeed :443` . No intentar "arreglar" el listener 80 de liteSpeed.

12. Estado final de los datos (DB)

DB productiva: `/root/induccion-megas/data/db.sqlite3` — SQLite, medidas el 06-oct después de aplicar migraciones (`SELECT count(*) FROM ...`).

Tabla	Filas
django_migrations	29
auth_user	4
auth_permission / auth_user_user_permissions	68 / 68
courses_course	2
courses_module	2
courses_lesson	11
courses_calificacion	2
courses_certificado	2
courses_userprogress	19
evaluaciones_evaluacion / _pregunta / _opcion	1 / 2 / 4
foro_thread / foro_post	4 / 2
django_admin_log / django_content_type / django_session	72 / 17 / 6
auth_group (+permissions)	0

(21 tablas de datos + ¿tablas de session? — el conteo de tablas de la BBDD es 21 sin incluir las `sqlite_*` internas; 22 contando `sqlite_sequence` .)

13. Pendientes para el equipo / usuario

- **Prueba manual en navegador** (final paso de QA): alta de cuenta nueva (debe estar el correo en `auth_user`), login, catálogo → lección → evaluación → generación de certificado (correo y descarga por token), foro, y acceso a `/admin/` .
- **No hace falta Firebase Console** para este flujo (verificado, sección 6.H). Solo si algún día se activa login por Google u OAuth se necesitará añadir `www.megas.com.co` a Authorized domains.
- **CalificacionesPage local sin Firestore**: la versión desplegada es la comiteada (usa Firestore del backend admin si procede). Si el equipo quiere la variante local, hay que comitearla y rebuildear.
- Los `context proxy` inactivos en los vhost.conf pueden limpiarse (estético), y conviene unificar/eliminar los `nginx-host.conf` / `frontend/nginx.conf` obsoletos para evitar confusiones.
- Migración futura de `/rutadeexpertos/` al stack nuevo está fuera del alcance de este día.

14. Apéndices

14.1 `filesParaDeploy/docker-compose.prod.yml` (estado final)

```
# ============================================================
#  docker-compose.prod.yml  —  SOLO para el VPS.
# ... (comentarios omitidos por brevedad; el archivo real los trae) ...
# ============================================================
name: induccion-megas

services:
  backend:
    build:
      context: ..
      dockerfile: Dockerfile.backend
    image: induccion-megas-backend:${APP_VERSION:-latest}
    container_name: ruta-induccion-backend
    restart: unless-stopped
    command: >
      sh -c "python manage.py migrate --noinput &&
             python manage.py collectstatic --noinput &&
             gunicorn --bind 0.0.0.0:8000 core_project.wsgi:application --timeout 120"
    ports:
      - "127.0.0.1:8001:8000"
    volumes:
      - /root/induccion-megas/data/db.sqlite3:/app/db.sqlite3
      - /root/induccion-megas/media:/app/media
    environment:
      - DEBUG=${DEBUG:-False}
      - SECRET_KEY=${SECRET_KEY:?falta SECRET_KEY en .env}
      - ALLOWED_HOSTS=${ALLOWED_HOSTS:-megas.com.co,www.megas.com.co}
      - CSRF_TRUSTED_ORIGINS=${CSRF_TRUSTED_ORIGINS:-https://megas.com.co,https://www.megas.com.co}
      - CORS_ALLOWED_ORIGINS=${CORS_ALLOWED_ORIGINS:-}
      - EMAIL_HOST_USER=${EMAIL_HOST_USER:?falta EMAIL_HOST_USER en .env}
      - EMAIL_HOST_PASSWORD=${EMAIL_HOST_PASSWORD:?falta EMAIL_HOST_PASSWORD en .env}
      - NOTIF_DESTINATARIOS=${NOTIF_DESTINATARIOS:?falta NOTIF_DESTINATARIOS en .env}
      - FIREBASE_PROJECT_ID=${FIREBASE_PROJECT_ID:?falta FIREBASE_PROJECT_ID en .env}
    logging:
      driver: json-file
      options:
        max-size: "10m"
        max-file: "3"

  frontend:
    build:
      context: ..
      dockerfile: Dockerfile.frontend
    image: induccion-megas-frontend:${APP_VERSION:-latest}
    container_name: induccion-megas-frontend
    restart: unless-stopped
    ports:
      - "127.0.0.1:5138:80"
    depends_on:
      - backend
    logging:
      driver: json-file
      options:
```

```
        max-size: "10m"
        max-file: "3"
```

14.2 `.htaccess` final (`/home/www.megas.com.co/public_html/.htaccess`)

```
RewriteEngine On

rewrite  {
  enable                  1
  autoLoadHtaccess        1
  rules                   <<
```

14.3 nginx del host — server block :80 final

```
server {
    listen       80;
    listen       [::]:80;
    server_name  _;

    # Webroot de ACME: lo comparte con LiteSpeed (renovacion del cert)
    location /.well-known/acme-challenge/ {
        root /usr/local/lsws/Example/html;
    }

    # Todo el trafico http va a https; el TLS lo termina LiteSpeed en 443
    location / {
        return 301 https://www.megas.com.co$request_uri;
    }
}
```

14.4 LiteSpeed — extprocessors relevantes (a nivel de servidor, `httpd_config.cor`

```
extprocessor dockerFrontend {
  type                    proxy
  address                 127.0.0.1:5137      # stack antiguo frontend (rutadeexpertos)
  maxConns                100
  pcKeepAliveTimeout      60
  initTimeout             60
  retryTimeout            0
  respBuffer              0
}
extprocessor dockerBackend {
  type                    proxy
  address                 127.0.0.1:8000      # stack antiguo backend
  maxConns                100
  pcKeepAliveTimeout      60
  initTimeout             60
  retryTimeout            0
  respBuffer              0
}
extprocessor dockerFrontend_Induccion {
  type                    proxy
  address                 127.0.0.1:5138      # stack nuevo frontend
```

```
  maxConns                100
  pcKeepAliveTimeout      60
  initTimeout             60
  retryTimeout            0
  respBuffer              0
}
extprocessor dockerBackend_Induccion {
  type                    proxy
  address                 127.0.0.1:8001      # stack nuevo backend
  maxConns                100
  pcKeepAliveTimeout      60
  initTimeout             60
  retryTimeout            0
  respBuffer              0
}
```

14.5 `nginx.conf` del CONTENEDOR frontend (raíz del repo)

```
# El TLS lo termina el nginx del host... se reenvia X-Forwarded-Proto.
map $http_x_forwarded_proto $forwarded_proto {
    default $http_x_forwarded_proto;
    ''      $scheme;
}

server {
    listen 80;
    server_name localhost megas.com.co www.megas.com.co;
    root /usr/share/nginx/html;
    absolute_redirect off;

    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $forwarded_proto;
    client_max_body_size 20M;

    location = / { return 302 /rutainduccion/; }
    location = /rutadeexpertos { return 301 /rutainduccion/; }
    location /rutainduccion { alias /usr/share/nginx/html/rutainduccion;
                              try_files $uri $uri/ /rutainduccion/index.html; }
    location /api/     { proxy_pass http://backend:8000; }
    location /admin/   { proxy_pass http://backend:8000; }
    location /static/  { proxy_pass http://backend:8000; }
    location /media/   { alias /usr/share/nginx/html/media/; }
    location /         { return 302 /rutainduccion$request_uri; }
}
```

14.6 Dockerfiles

Dockerfile.backend (python:3.12-slim):

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
RUN python manage.py collectstatic --noinput
EXPOSE 8000
```

Dockerfile.frontend (Node 20 → nginx:alpine):

```
FROM node:20-alpine AS build
WORKDIR /app
COPY frontend/package*.json ./
RUN npm install
COPY frontend/ ./
RUN npm run build

FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html/rutainduccion
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

14.7 Firebase (referencia pública, sin secretos)

```
frontend/src/config/firebase.js  (valores vigentes, incrustados en el bundle desplegado)
  apiKey:            "AIzaSyDFjt0nBYpyVXEFkti8CugjSS5DBwoFI18"   # key pública (va en el bundle)
  authDomain:        "plataforma-gmax.firebaseapp.com"
  projectId:         "plataforma-gmax"
  storageBucket:     "plataforma-gmax.firebasestorage.app"
  messagingSenderId: "213219499650"
  appId:             "1:213219499650:web:b1f5717ece4268c2111e9b"
Uso: interno en el backend mediante Firebase Admin (FIREBASE_PROJECT_ID) para auditar ID tokens.
```

> La API key de Firebase es pública por diseño (nunca es un secreto). Las credenciales que NO se de
> SECRET_KEY de Django y EMAIL_HOST_PASSWORD.

14.8 Referencia rápida de comandos del día (para reproducir)

```
# Local (Windows) — push de los 4 commits a segundaSimple
git push origin segundaSimple

# VPS — despliegue backend+frontend, con APP_VERSION
cd /root/induccion-megas
git fetch origin && git checkout segundaSimple && git pull
export APP_VERSION=$(git rev-parse --short HEAD)
docker build --network host -t induccion-megas-backend:$APP_VERSION  -f Dockerfile.backend  .
docker build --network host -t induccion-megas-frontend:$APP_VERSION -f Dockerfile.frontend .
docker compose --env-file .env.produccion -f filesParaDeploy/docker-compose.prod.yml up -d --no-deps ba
docker compose --env-file .env.produccion -f filesParaDeploy/docker-compose.prod.yml up -d --no-deps fr
docker exec ruta-induccion-backend python manage.py migrate --noinput

# VPS — routing (ya aplicado en el día): editar .htaccess; nginx host ya redirige a https.
nginx -t && systemctl reload nginx
/usr/local/lsws/bin/lswsctrl restart        # solo si se cambia liteSpeed (no para .htaccess)
```

Fin del documento. Reporte elaborado el 06-oct-2026 a partir de la bitácora de despliegue del stack nuevo de la Ruta de Inducción (rama segundaSimple). No contiene credenciales sensibles.